BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

## Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

### Page 1 onwards · Do Now

Q1. An existing node reference.

Q2. Its first node.

Q3. Left subtree keys less and right keys greater under chosen policy.

Q4. BST root10, left6, right18. Insert 15. Give location.

### You Do pages · Independent questions

Q1. 6<8 [2]; 6>3 [2]; right child of3 [2].

Q2. 3,6,8,10 [4]; left-root-right order follows BST invariant [2].

Q3. Balanced height ≈log n [2]; follow only one path per comparison [1]; sorted insertions may create chain height n [2]; search may inspect n nodes [1].

### Exit Ticket · Model answer

False. It can become a chain.

## Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

**1. Copy the whole original question:**

**2. Write a complete new answer in GREEN PEN. Show working if needed:**



RHS COMPUTING / OCR H446 / mixed LESSON 052 · DRAFT

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

# LESSON 052 · Lesson 052 · Binary Search Trees with Node Objects

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

## Do Now · Recall previous learning

Q1. What does LinkedList<T>.AddAfter use to place a new node?

Q2. What does HEAD mean for a linked list?

Q3. What is an invariant of a BST?

Q4. 022

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

## I Do · Read new knowledge

### Key vocabulary

BST — Binary search tree ordered by key.

root — Topmost tree node.

leaf — Node without child nodes.

inorder — Left subtree, root, right subtree.

### New learning

Discover — A binary search tree (BST) keeps smaller keys in the left subtree and larger keys in the right subtree under a consistent duplicate-key policy.

Trace — Each node stores a key and optional Left/Right child references. Inserting begins at the root and follows comparisons until an empty child link is found.

Evaluate — Inorder traversal (left, root, right) prints sorted keys for a valid BST without duplicates.

### Worked example

Insert key 7 into BST root 8, left 3, right 10, with 6 right of 3.

1. Explain the trace — 7 becomes right child of 6; follow comparisons until a null child is reached.

## We Do · Practise with guidance

Insert 12 into a different BST: root 9, children 5 and 14.

```
12>9 → right(14)
12<14 → left child of 14
```

Explain the trace

Place 12 as left child of 14. Tree ordering remains valid.





BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

## You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

### Question 1

BST root8, left3, right10, insert 6. Show comparisons and location. [6]

### Question 2

For BST with keys 8,3,10,6, give inorder traversal and explain sorting. [6]

### Question 3

Explain why BST search may be O(log n) when balanced but O(n) when skewed. [6]

## Exit Ticket · Answer before checking

An unbalanced BST is guaranteed to have logarithmic search time.